Lucius Hatherly Retina Kaggle Report

Kaggle Team Name: ift6390_Lucius_Hatherly_20294880

Overview: This report presents my approach in developing a machine learning algorithm to predict retinal diabetic retinopathy from retinal fundus photographs. This project utilizes feature engineering, preprocessing, and machine learning optimization. Note the models explored in the reports generate two main sets of submission, one prior to November 30th (where sklearn models were prohibited) and one after November 30th (where sklearn models and tensor flows were permitted)**.**

1. Feature Design:

Data Cleaning & Preparation: Prior to training our models, I created a multi-stage preprocessing and feature engineering pipeline which guaranteed that the inputs were informative, reliable, and properly suitable for classical machine learning models.

Data Integrity & Structure: Firstly, for both milestones, we loaded the training and test datasets derived from the assigned pickle files. Note each of these files possessed RGB retinal images and the training data set also contained the corresponding ordinal diabetic-retinopathy labels. The initial data exploration confirmed that the images were stored with the structure as 28×28×3 arrays of type uint8, with 2352 raw pixel features, and having intensities on a scale from 0 to 255. Subsequently, I proceeded to verify the missing or invalid values for both the image arrays and the labels. The np.isnan pixel inspection revealed that neither the training nor test sets had missing values.

Considering that diabetic retinopathy impacts blood vessel color, shape, and microvascular lesions, it is important to retain the entire RGB data set. Choosing to convert the RGB data to grayscale would remove medically important information. Moreover, my data exploration confirmed that the label distribution among the five disease-severity classes helped me understand class imbalance and verify random samples visually to validate the consistency and quality of the data. This encouraged me to carefully consider how to handle the train_test_split later.

Outlier Removal and Brightness: I computed the total pixel intensity and visualized the generated distribution. The two heavy tails in the image revealed over exposed (too bright) or under exposed (too dark) outliers. Given that these outliers impact the retinal structure, removing these outliers could improve the performance of distance-based models like Logistic Regression and KNN [1]. I then proceeded to remove outlier images based on the overall brightness of the image. The outlier removal was performed through computing the sum of all pixel intensities and removing extremely bright outliers through discarding the lowest 0.5% and highest 0.5% of this distribution.

Low Variance Image Filtering: After the brightness-based filtering, I applied a second cleaning step to filter out extremely low variance images. This is because these images usually contain little to no information on the retinal structure because of problems like blur, severe underexposure, or possibly acquisition artifacts. I calculated the pixel-intensity variance for each image and removed images with a variance lower than the first percentile. Note that these two cleaning steps were performed before any reshaping or normalization to ensure that variance and brightness calculations were conducted on raw, spatially intact image tensors without any scale distortions.

Flattening and Normalization: Following data cleaning, we prepared the dataset for classical ML models by reshaping each 28×28×3 image into a 2D feature vector of length 2352. Pixel normalization was then applied by scaling all values to the [0, 1] range through division by 255. This flattening and normalization ensures that the gradient stabilizes for logistic regression and inhibits large magnitude features from dominating models like KNN [2].

Results of Feature Engineering: The outlier removal of bright and dark images along with low variance filtering improved validation accuracy for logistic regression by around 3%. This demonstrates that clean and high contrast retinal images provide more discriminative pixel-level structure. These improvements justified using the cleaned dataset for all post-November-30th (MPP) experiments.

However, the KNN and SVM models declined on Kaggle by 2 to 3% with the feature engineering steps, which removed images by brightness and high variance. This is possibly because these non-parametric models rely heavily on the global geometry and density of the dataset. Although some filtered images may be extreme in brightness or contrast, these images possibly still contain important boundaries or support-vector information. Eliminating these data points seems to change neighborhood structure (for KNN) and margin-defining samples (for SVM). Moreover, it is possible the evaluated Kaggle data set may contain these 'outlier' images, meaning the filtered models receive a distribution shift. In conclusion, although feature cleaning improves linear models like Logistic Regression, it can reduce generalization for distance-based or margin-based classifiers

Consideration Point: Note that for the models from the pre-November 30th milestone, I performed two types of data preprocessing. For the first method, I investigated missing values, flattened the image data, and normalized the train and test image data. For the second method, I performed these steps in addition to removing outliers (specifically for excessively bright and low variance images). So effectively, the second method had the same feature preprocessing and data transformation steps for after November 30th (milestone 2). Note the goal of the submissions for the first milestone was focused on beating the Kaggle baselines rather than refined feature engineering and optimizing the model as much as possible like milestone 2.

2. Algorithms & Methodology:

Data Splitting: For both the two milestones, I divided the dataset into training (80%) and validation (20%) subsets to ensure that the quality score was properly maintained across the data set. Note for the model testing prior to November 30th where sklearn was prohibited, I created a custom train_test split function which replicated sklearn behaviour to split the data into train and validation data sets. For data splitting after November 30th, I used the sklearn train_test_split function with stratified sampling to produce my train and validation set for the retinal data.

Model Selection: 1. For the first milestone, I created custom KNN, Logistic Regression, and SVM models on the data each with hyperparameter tuning. All these methods were successful in passing the Kaggle baseline with Logistic Regression model performing the best in Kaggle. These models were selected as they could all be created with numpy and custom functions without having to use sklearn and could serve as good classification methods to predict the relevant labels for the retinal images. 2. After I beat

the Kaggle baseline in Milestone 1, I expanded the modeling suite for Milestone 2 to include more powerful classifiers via the new scikit-learn and TensorFlow libraries. I evaluated four algorithms: Logistic Regression, Random Forests, Multi-Layer Perceptrons (MLPs), and Convolutional Neural Networks (CNNs). I selected sklearn Logistic Regression as a strong linear baseline, particularly well-suited for high-dimensional pixel data [2]. Random Forests were chosen as a non-parametric ensemble method to realize the non-linear decision boundaries and feature interactions [3].

The MLP evaluated served as a fully connected neural model, which possessed the ability to learn nonlinear pixel patterns post normalization. The MLP I designed used a 2352-dimensional flattened input layer and hidden layers of size 64 or 128. Finally, I implemented a compact CNN to use the retinal spatial structure directly. The CNN was broken down into 3 convolutional blocks each with increasing filters (32 to 64 to 128), Batch Normalization, MaxPooling, and a Dense(128) head with dropout. All these additional hyperparameters were added to optimize CNN's performance and ensure it performed as strong as possible.

Regularization: Note Milestone 1 focused less on regularization as its key goal was to beat the Kaggle baseline, and instead regularization was largely performed in Milestone 2. Regularization was key in maintaining the performance and stability of several models. Logistic Regression used L2 regularization and the lbfgs optimizer to minimize overfitting and ensure better optimization within the high-dimensional 2352-feature space [4]. Random Forests implicit regularization using hyperparameters like max_depth, min_samples_split, min_samples_leaf, and class_weight [5]. These hyperparameters constrain tree growth to limit overly complex decision boundaries [6].

Regularization was key to the success of the neural models. MLP utilized L2 weight decay and early stopping to reduce high variance growing within limited data [7]. CNN will employ Batch Normalization throughout each convolutional block to maintain training and Dropout(0.4) within its dense layer to reduce overfitting [8]. The neural model also depended on learning-rate reduction to implicitly regulate optimization dynamics [9].

Hyperparameter & Optimization: To optimize the model's performance for Milestone 1, I performed thorough hyperparameter searches for KNN, Logistic Regression, and SVM where I utilized only NumPy-based implementations. With respect to KNN, I tested a wide range of odd values of k (3–31) using L2-normalized cosine similarity and then proceeded to select the k value which optimized validation accuracy. For Logistic Regression, we performed a grid search over multiple learning rates (0.001–0.01) and iteration limits (3,000–12,000), retraining each model on the training split and selecting the hyperparameter pair that achieved the highest validation accuracy.

Lastly, with respect to the multiclass SVM, I tested combinations of learning rates, regularization strengths C, along with training epochs via iterating through the parameter sets and calculating the validation accuracy across every trained model. After I evaluated the three algorithms hyperparameter combinations, the best-performing hyperparameters were chosen based on validation accuracy, and these three models were then retrained using the full training set to produce the appropriate Kaggle submission predictions.

For Milestone 2, each model went through structured hyperparameter tuning process specified to its complexity. Logistic Regression was tuned via a grid search over regularization strengths $C \in \{0.001, 0.01, 0.1, 1\}$ along with iteration limits (3000–10,000). The best-performing Logistic Regression produced a validation accuracy of around 51.5%. The Random Forest hyperparameter tuning saw a grid search throughout its key structural hyperparameters, including the tree number, maximum depth, maximum feature sampling strategy (sqrt vs. log2), and class-weight options. Lastly, the Random Forest produced strong validation accuracy, its generalization to Kaggle was poor as it failed to beat the baseline.

Moreover, the MLP model was tuned using a grid across hidden layer sizes of (64 or 128 units), initial learning rates (0.001), L2 penalty strengths, along with iteration limits. The early stopping was used to stop training once the validation performance had plateaued. I elected to not use a proper grid search for CNN due to computational constraints; rather my CNN optimization focused on selecting key architectural components. The architectural components selected included the Adam optimizer with learning-rate scheduling, ReduceLROnPlateau for dynamic adjustment of learning rates, along with EarlyStopping to preserve the best-performing weights. The training for CNN was performed up to 25 epochs along with batch sizes of 32.

Model Evaluation, Performance Metrics, and Error Analysis: For Milestone 1 models, the main metric used to evaluate model performance was classification accuracy both in Python and in Kaggle. Given that the main goal for the model testing prior to November 30th was to beat the Kaggle baseline, there was no confusion matrix and error analysis conducted to gain further understanding of the models.
For Milestone 2, I performed more extensive model evaluation using metrics and error analysis. These metrics included classification reports (precision, recall, F1-score per class) and confusion matrices. I also calculated the validation error (1-accuracy) for the best performing hyper parameter combination across every model. The classification report and confusion matrix analyses displayed common patterns. Specifically, class 0 ("no DR") was far easier to identify due to its dominance compared to other classes in the data set. This contrasts with the minority classes (1–4) suffering from smaller recall and precision. Unfortunately, this class imbalance increases the label prediction difficulty for classical ML models and neural networks.

3. Results:

Model	Milestone	Preprocessing Used	Val Accuracy (Python)	Kaggle Accuracy
KNN (k=11)	M1	No outlier removal	**0.463**	**0.465**
Logistic Regression (custom)	M1	Outlier removal + low-variance filter	**0.4764**	**0.51**
SVM (custom)	M1	No outlier removal	**0.4537**	**0.425**
Logistic Regression (sklearn OVR)	M2	Outlier removal + low-variance filter	**0.4528**	**0.515**
Random Forest (sklearn)	M2	Outlier removal + low-variance filter	**0.4623**	**0.445**
MLP (sklearn)	M2	Outlier removal + low-variance filter	**0.4575**	**0.475**
CNN (TensorFlow)	M2	No feature removal (raw RGB, normalized)	**0.4491**	**≈0.4950**
CNN (TensorFlow)	M2	Feature removal (raw RGB, normalized)	**0.4528**	**≈0.49**

Figure A: Table Summarizing Model Performance across Milestone 1 and 2

For the first milestone, I trained and then evaluated three NumPy-based machine learning models—K-Nearest Neighbors (KNN), Multiclass Logistic Regression, and Multiclass SVM—using the processed and normalized retinal image dataset. KNN was tuned across a range of odd *k* values (3–31), and the best validation accuracy was achieved at **k = 11** with an accuracy of **0.4630**, which also produced a Kaggle public leaderboard score of **0.4650 (note for this best performing KNN, no outlier images were removed)**. Logistic Regression was trained using a learning rate of 0.01 and 12,000 iterations, achieving a validation accuracy of **0.4764** and a Kaggle score of **0.50 (this best performing Logistic Regression model had outlier images removed).** For SVM, we performed a grid search over learning rates, regularization strengths, and epoch counts; the best configuration obtained **0.4537** validation accuracy and a Kaggle score of **0.425 (this best performing SVM had no outlier images removed)**. Across all methods, Logistic Regression demonstrated the strongest performance on the Kaggle leadership board, followed by KNN and then SVM. These results established a strong foundational benchmark for moving into higher-capacity models permitted in Milestone 2.

In Milestone 2, the four additional models evaluated using scikit-learn and TensorFlow were Logistic Regression (OVR), Random Forest, Multi-Layer Perceptron (MLP), and a compact CNN. I further stratified an 80/20 train–validation split to maintain class balance. Surprisingly, Logistic Regression remained the best performing model. Logistic Regression produced a python validation accuracy of 0.4528 and the best performing Kaggle score among all Milestone 2 models at 0.515 post tuning the hyperparameters: C, max_iter, and class_weight. The Random Forest model achieved a higher Python validation accuracy (0.4623), however, the model generalized poorly to Kaggle (0.445). This implies that Random Forest model exhibited overfitting and sensitivity possibly due to the removal of bright or low-variance images. Moreover, the MLP achieved a validation accuracy of 0.4575 and produced a Kaggle score of 0.475. This relatively lower score for this expressive model was limited by the small dataset and class imbalance.

The two CNN models trained with and without feature removal, yielded python validation accuracies between 0.4491–0.4528, and Kaggle accuracy scores around 0.49–0.495. These models did beat the Kaggle baseline, however, they were unsuccessful in bettering the performance of Logistic Regression. This is likely because of the small size of the image dataset size (≈1,068 images) and the general challenges about training a CNN from the ground up for subtle retinal patterns without any transfer learning being performed. Across both milestones, Logistic Regression emerged as the most reliable and highest-performing model overall. The evaluated nonlinear models like Random Forest and MLP achieved comparatively good python validation scores, but these scores did not consistently translate to high Kaggle accuracy possibly due to overfitting and sensitivity to preprocessing choices. The Random Forest model is likely not sufficiently complex to capture the image feature data set. In conclusion, these results demonstrate that on small medical-image datasets without transfer or pretrained learning, the well-regularized linear models can perform even better than deeper architectures. This seems to be especially possible when we combine preprocessing such as brightness filtering, variance-based cleaning, and normalization which reduce accuracy for certain models.

4. Discussion:

Milestone 1 Discussion

The classical ML models used in Milestone 1 reflect both the structure of the retinal fundus dataset and the limitations of linear and distance-based classifiers when directly applied to raw pixel data. The KNN model achieved a Kaggle accuracy of 0.465, benefiting from normalization and cosine similarity.

However, the performance was constrained by the curse of dimensionality and the sensitivity to noise in flattened 2352-dimensional vectors. Logistic Regression performed best in Milestone 1 with a Kaggle score of 0.51. This performance is consistent with the ability to learn smoother global decision boundaries that properly capture pixel-intensity trends which correspond with the retinal disease level. In contrast, the SVM model performed poorly. It reached only 0.425 on kaggle, likely because of not using nonlinear kernels. Without kernels, SVM was restricted to a much weaker hypothesis class and showed unstable training for the high dimensional image training space. Feature preprocessing impacted the models differently. Filtering out extreme-brightness and low-variance outliers marginally improved Logistic Regression Kaggle from 0.5 to 0.51 likely by reducing noise and stabilizing gradient descent. However, KNN and SVM saw performance drops. I speculate this is because the geometry of the data: removing even a small number of outliers shifts the neighborhood distances in KNN or alters support vectors used in SVM, leading to worse generalization.

In Milestone 2, using scikit-learn and TensorFlow allowed more expressive models to be evaluated. These included Random Forests, MLPs, and CNNs. The main pattern from Milestone 1 continued: Logistic Regression remained the strongest performing model and achieved the highest Kaggle accuracy of 0.515. The Logistic Regression model outperformed deeper neural models. These results suggested that for small retinal-image datasets, simple linear models with good regularization generalize better than theoretically high performing neural models. The Random Forests demonstrated a higher validation accuracy (0.4623) than Logistic Regression, however, these models generalized poorly to Kaggle (0.445). This poor Random Forest Kaggle score implies overfitting, likely driven by the limited dataset size and the model's sensitivity to the earlier outlier-removal steps. These removals altered pixel-level variance that tree-based models use for splits. The MLP achieved moderate Kaggle performance (Kaggle ~~0.475), likely derived from nonlinear feature learning, however this low score was likely driven by the small dataset and class imbalance. The CNN models produced Kaggle accuracies around 0.49–0.495, consistently beating the baseline but low compared to Logistic Regression. The primary limiting factors were dataset size (~1,068 images), the subtlety of the diabetic retinopathy lesions at 28×28 resolution, and the absence of transfer learning. The CNNs models typically need much more data to surpass classical ML models and properly analyze non-linear hierarchical data.

Overall Discussion

Throughout the two milestones, this key pattern emerged: model complexity did not correlate with improved performance. Logistic Regression—a simple linear classifier—outperformed all other models in both Milestone 1 and Milestone 2. This allows me to conclude that small, high-dimensional datasets benefit from strong regularization more than from model depth. Moreover, outlier removal helps linear models but negatively impacts tree and distance-based models. This indicates that preprocessing choices must consider the behavior of the model family. CNNs and other neural models require either more data or transfer learning to outperform classical models in medical-image tasks [10].

The experiments demonstrate that for small clinical imaging datasets, well-regularized linear models—combined with careful preprocessing—can outperform deeper architectures, highlighting the importance of tailoring model choice to dataset size, structure, and available computational resources. The limitation of my project was not performing more data augmentation of pretraining when testing my CNN model, it would be interesting to investigate in future work if any of these steps would enable CNN to surpass Logistic Regression as the highest performing model.

5. References:

[1] García, S., Luengo, J., & Herrera, F. *Data Preprocessing in Data Mining*. Springer, 2015, [pp 87-94].

[2] Géron, Aurélien. *Hands-On Machine Learning with Scikit-Learn*, Keras & TensorFlow, 2nd Edition. O'Reilly Media, 2019, [pp 68-70].

[3] Breiman, Leo. *"Random Forests." Machine Learning 45*, no. 1 (2001), [pp 5–32].

[4] Ng, Andrew. *"Feature Selection, L1 vs. L2 Regularization."* Stanford CS229 Lecture Notes, 2004, [pp 3-6].

[5] Hastie, Tibshirani & Friedman — *The Elements of Statistical Learning*, 2nd Edition, Springer, 2009, [pp 587-592].

[6] Ibid.

[7] Goodfellow, I., Bengio, Y., & Courville, A, *Deep Learning*. (2016), [pp 225-236]

[8] Ioffe, S., & Szegedy, C. (2015). *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift*. Proceedings of the 32nd International Conference on Machine Learning (ICML), [pp. 448–456].

[9] Goodfellow, I., Bengio, Y., & Courville, A. (2016), *Deep Learning*. MIT Press *Chapter 8: Optimization for Training Deep Models*, [pp. 287–292].

[10] Litjens, G., Kooi, T., Bejnordi, B. E., et al. (2017), *A Survey on Deep Learning in Medical Image Analysis, Medical Image Analysis*, [pp 42, 60–88].

6. Appendix:

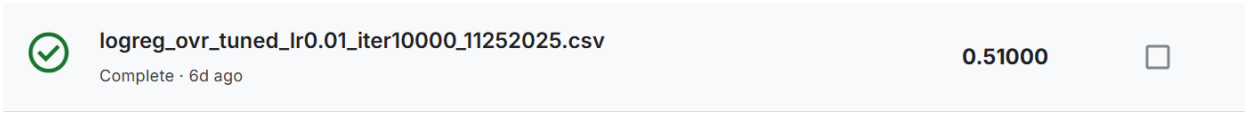

Figure B: Best Performing for Logistic Regression Kaggle Submission Milestone 1

Figure C: Best Performing for KNN Kaggle Submission Milestone 1

Figure D: Best Performing SVM Kaggle Submission Milestone 1

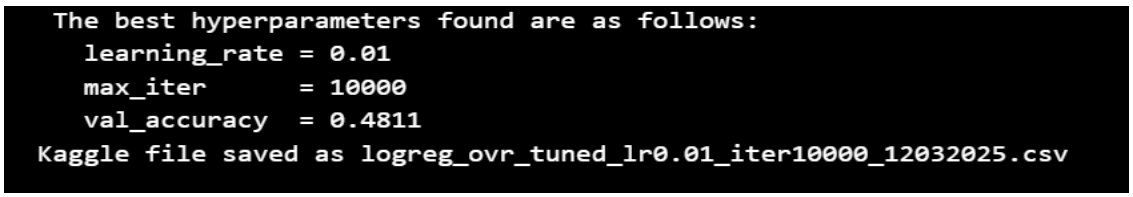

Figure E: Best Performing Logistic Regression Parameters Milestone 1

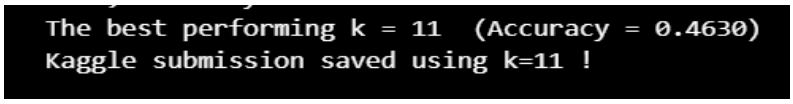

Figure F: Best Performing KNN Validation Accuracy Milestone 1

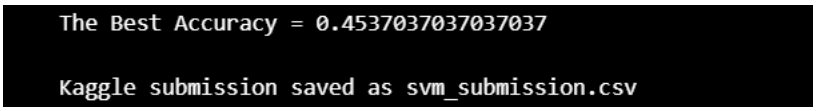

Figure G: Best Performing SVM Validation Accuracy Milestone 1

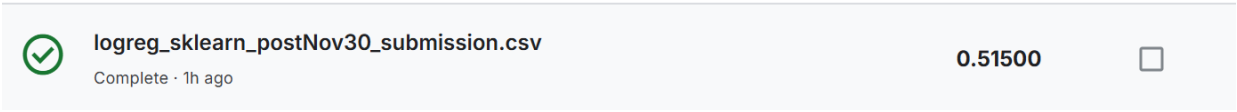

Figure H: Best Performing Logistic Regression Model for Kaggle Submission Milestone 2

Figure I: Best Performing Random Forest Model for Kaggle Submission Milestone 2

Figure J: Best Performing MLP Model for Kaggle Submission Milestone 2

Figure K: Best Performing CNN Models for Kaggle Submission Milestone 2

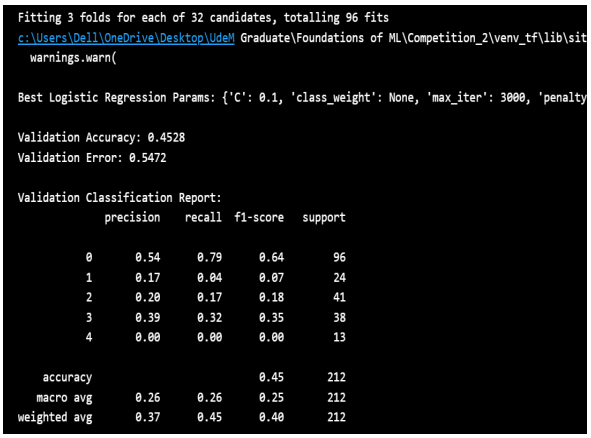

Figure L: Classification report for Logistic Regression Milestone 2

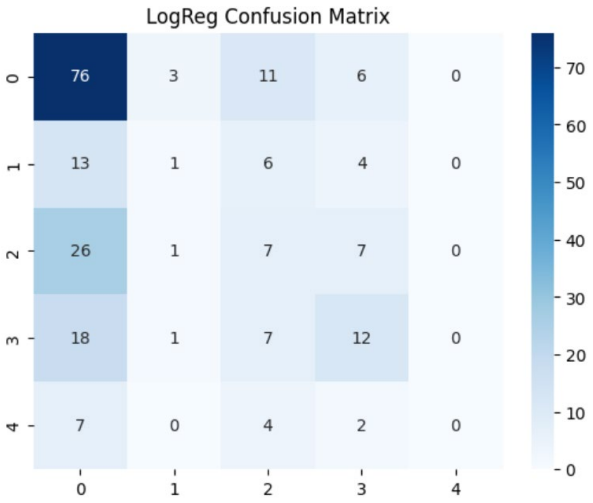

Figure M: Confusion Matrix for Logistic Regression Milestone 2

```
The Random Forest Grid is running
Fitting 3 folds for each of 144 candidates, totalling 432 fits

Best Random Forest Params: {'class_weight': None, 'max_depth': 20, 'max_features':

The Validation Accuracy is: 0.4623
The Validation Error is:   0.5377

The Validation Classification Report is:
              precision    recall  f1-score   support

           0       0.62      0.75      0.68        96
           1       0.42      0.21      0.28        24
           2       0.27      0.29      0.28        41
           3       0.23      0.24      0.23        38
           4       0.00      0.00      0.00        13

    accuracy                           0.46       212
   macro avg       0.31      0.30      0.29       212
weighted avg       0.42      0.46      0.43       212
```

Figure N: Confusion Matrix for Random Forest Milestone 2

```
MLP Grid Search is running
Fitting 3 folds for each of 6 candidates, totalling 18 fits

The Best MLP Params: {'alpha': 0.0001, 'hidden_layer_sizes': (128,),

Validation Accuracy: 0.4575
Validation Error:   0.5425

Validation Classification Report:
              precision    recall  f1-score   support

           0       0.62      0.77      0.69        96
           1       0.33      0.08      0.13        24
           2       0.18      0.17      0.18        41
           3       0.31      0.29      0.30        38
           4       0.23      0.23      0.23        13

    accuracy                           0.46       212
   macro avg       0.34      0.31      0.31       212
weighted avg       0.42      0.46      0.43       212
```

Figure O: Confusion Matrix for MLP Milestone 2

```
Training CNN
Epoch 1/25
27/27 - 6s - 208ms/step - accuracy: 0.4028 - loss: 1.4005 - val_accuracy: 0.4491
Epoch 2/25
27/27 - 2s - 72ms/step - accuracy: 0.4861 - loss: 1.2563 - val_accuracy: 0.4491
Epoch 3/25
27/27 - 2s - 71ms/step - accuracy: 0.4988 - loss: 1.2168 - val_accuracy: 0.4491
Epoch 4/25
27/27 - 2s - 73ms/step - accuracy: 0.4931 - loss: 1.2212 - val_accuracy: 0.4491
Epoch 5/25
27/27 - 2s - 74ms/step - accuracy: 0.5058 - loss: 1.1815 - val_accuracy: 0.4491
Epoch 6/25
27/27 - 2s - 75ms/step - accuracy: 0.5231 - loss: 1.1492 - val_accuracy: 0.4491
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step

Validation Accuracy: 0.44907407407407407

Error: 0.5509259259259259
```

Figure P: Confusion Matrix for CNN for Milestone 2

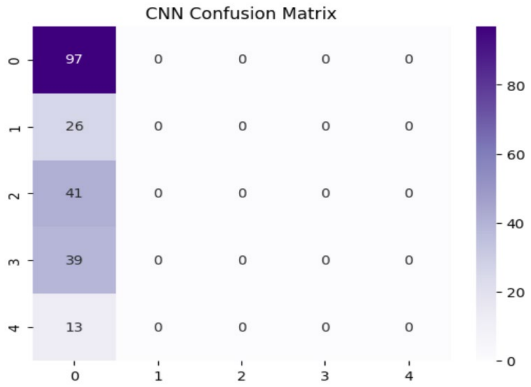

Figure P: Confusion Matrix for CNN for Milestone 2

Best Logistic Regression Params: {'C': 0.1, 'class_weight': None, 'max_iter': 3000, 'penalty': 'l2', 'solver': 'lbfgs'}

Figure Q: Best Parameters for Logistic Regression for Milestone 2

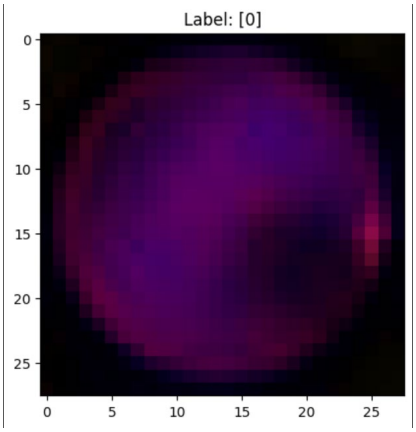

Figure R: Visualization for Image Data